Provable Security

William Bardon Charles Nicholas

Computer Science and Electrical Engineering Department, UMBC

Problem Statement

Generative AI code production exceeds the capacity for human review.

Formal Methods

Strengths	Weaknesses
Proof is absolute	Proving is slow
Tools exist	Tool syntax hard to learn
	Non-answer possible

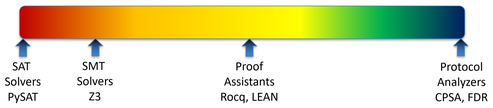


Figure 1:Spectrum of Formal Methods Tools

Failure Cases

- Resource Exhaustion
  - Memory
  - Storage
  - Time
- Model Syntax Error
- Undecidable
  - Insufficient information (many possible solutions)
  - Contradictory Constraints (no solution)

Generative AI and LLMs

Strengths	Weaknesses
Fast response	Training resources (time, data, hardware)
Usually correct	May hallucinate
Interpolating	Extrapolating
from examples	

Proposed Hybrid Architecture

- Formal Methods Tools
- Generative AI and Large Language Models (LLMs)

Heuristic Code Tools

- Compilers
- Linters
- Static Code Analyzers (SAST)

Design by Contract

- Detailed Code Comments
- Assertions in code for pre- and post-conditions

Test-Driven Development

- Representative Test Cases
- Both positive and negative tests
- Descriptive failure output

Simplification and Subdivision

Simplification may include

- Dimension reduction
- Polynomial approximation across a finite range
- Range segmentation
- Assumptions about values that allow symbolic simplifications in formulas

Subdivision solves for a specific value or range of values, but must be combined to address the entire problem.

Meta-Specification

is minimum information needed to generate:

- Specification text
- Verifier Model
- Program Code and tests

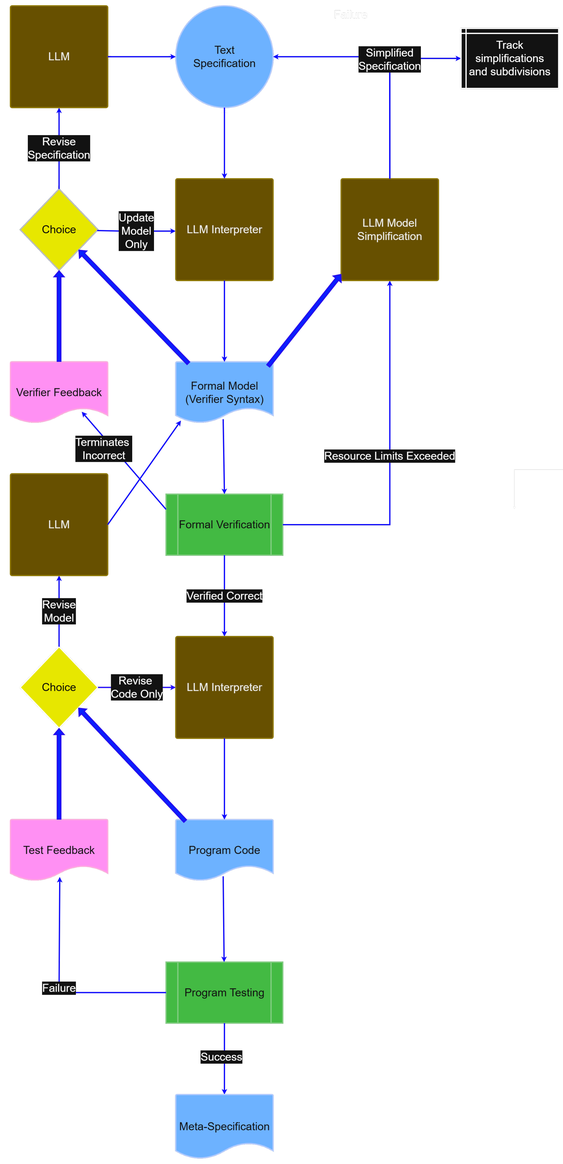


Figure 2:Production Cycle

Accepted by the 2026 Malware Technical Exchange Meeting

Progress

- Proof of concept translating logical statements in Z3 SMT Prover Python API.

Future Work

- Train specialized AI Models
- Build Application Framework
- Ingest IETF Specifications
- Infer Specification from Existing Implementation Code

References

[1] Counterexample Guided Inductive Synthesis Using Large Language Models and Satisfiability Solving, Sumit Kumar Jha et al., IEEE, https://doi.org/10.1109/MILCOM58377.2023.10356332, 2023

[2] Assigning Meanings to Programs, Robert W. Floyd, https://people.eecs.berkeley.edu/~necula/Papers/FloydMeaning.pdf, 1967

[3] An Axiomatic Basis for Computer Programming, C.A.R. Hoare, ACM, https://doi.org/10.1145/363235.363259, 1969

[4] Beyond Memorization: Testing LLM Reasoning on Unseen Theory of Computation Tasks, Shlok Shelat et al., Arxix, https://doi.org/10.48550/arXiv.2601.13392, 2026

[5] Applying 'Design by Contract', Bertrand Meyer, https://doi.org/10.1109/2.161279, 1992

[6] Test-Driven Development: By Example, Kent Beck, Addison-Wesley, https://dl.acm.org/doi/abs/10.5555/579193, 2015

Contact Information

- Email:
  b277@umbc.edu
  nicholas@umbc.edu

UMBC